PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL
ESCOLA POLITÉCNICA
Trabalho 1 - Projeto e Otimização de Algoritmos

DHARA HADI
THOMAZ GOMES SZECKIR

MAIOR SUBSEQUÊNCIA COMUM (LCS)

Algoritmo recursivo (força bruta) e Programação Dinâmica

Porto Alegre
2026

1 DESCRIÇÃO DO PROBLEMA

1.1 O que é uma subsequência

Uma subsequência de uma string é o que sobra quando apagamos alguns caracteres dela (ou nenhum), sem mudar a ordem dos que ficaram. Por exemplo, `ACE` é subsequência de `ABCDE`, porque basta apagar o B e o D. Já `AEC` não é, porque o C vem antes do E na string original.

1.2 A maior subsequência comum

Uma subsequência comum de X e Y é uma string que é subsequência das duas ao mesmo tempo. O problema da LCS é descobrir o comprimento da maior delas.

Um exemplo clássico é X = `ABCBDAB` e Y = `BDCABA`. Nesse caso, `BCBA`, `BDAB` e `BCAB` aparecem nas duas strings e têm tamanho 4, e não existe nenhuma subsequência comum maior do que isso. Então L = 4. O exemplo também mostra que pode haver mais de uma LCS, mas o tamanho é sempre o mesmo.

Alguns casos são fáceis de prever: se uma das strings é vazia, L = 0; se as duas são iguais, L = m; e se não têm nenhum caractere em comum, L = 0. Como a comparação é feita caractere por caractere, maiúsculas e minúsculas contam como diferentes, e o espaço conta como um caractere normal.

2 EXEMPLOS DE CASOS DO PROBLEMA

Na Tabela 1 estão alguns exemplos que escolhemos para testar o programa. Tentamos cobrir tanto situações comuns quanto os casos-limite. Todos eles fazem parte da bateria de testes descrita na Seção 5.5.

Tabela 1 - Exemplos de casos do problema da LCS

Caso	S1 e S2	L	Uma LCS	Situação
01	`ABCBDAB` `BDCABA`	4	`BCBA`	exemplo clássico; BDAB e BCAB também servem
02	`AGGTAB` `GXTXAYB`	4	`GTAB`	caracteres da LCS intercalados
03	`ALGORITMO` `ALGORITMO`	9	`ALGORITMO`	strings iguais
04	`ABCDEF` `GHIJKL`	0	`(vazia)`	nada em comum
05	`(vazia)` `PUCRS`	0	`(vazia)`	uma string vazia
06	`ALGORITMO` `LOGARITMO`	7	`LGRITMO`	palavras parecidas
08	`PROGRAMACAO DINAMICA` `PROJETO DE ALGORITMOS`	8	`PROO DIM`	frases com espaço
09	`ACCGGTCGAGTGCGCGGAAGCCGGCCGAA` `GTCGTTCGGAATGCCGTTGCTCTGTAAA`	20	`GTCGTCGGAAGCCGGCCGAA`	sequências de DNA

O caso 01 é um exemplo clássico, que tem três LCS diferentes de tamanho 4 (o programa devolve `BCBA`). No caso 02, os caracteres que formam a LCS aparecem misturados com outros que não entram nela. Os casos 03, 04 e 05 são os casos-limite. O caso 06 é curioso: ALGORITMO e LOGARITMO têm praticamente as mesmas letras, mas como algumas estão em posições trocadas, só 7 das 9 conseguem ser aproveitadas. Já no caso 08 a LCS encontrada tem um espaço no meio, o que mostra que o espaço é tratado como qualquer outro caractere.

3 ALGORITMO RECURSIVO (FORÇA BRUTA)

3.1 Ideia e recorrência

A forma mais ingênua de resolver o problema seria gerar todas as 2^m subsequências de X e testar uma por uma se ela também é subsequência de Y. Isso funciona, mas é muito lento. Uma forma mais inteligente de pensar a força bruta é olhar só para o último caractere de cada string.

Vamos chamar de c(i, j) o tamanho da LCS entre os i primeiros caracteres de X e os j primeiros caracteres de Y. Se um desses prefixos for vazio, não tem o que comparar e c(i, j) = 0. Se os últimos caracteres forem iguais ($x_i = y_j$), faz sentido usar esse caractere na LCS e continuar o problema sem ele nas duas strings, então c(i, j) = c(i - 1, j - 1) + 1. Se forem diferentes, pelo menos um dos dois não entra na LCS. Como não sabemos qual, testamos as duas opções (tirar o último de X ou tirar o último de Y) e ficamos com a que der o maior resultado.

3.2 Algoritmo

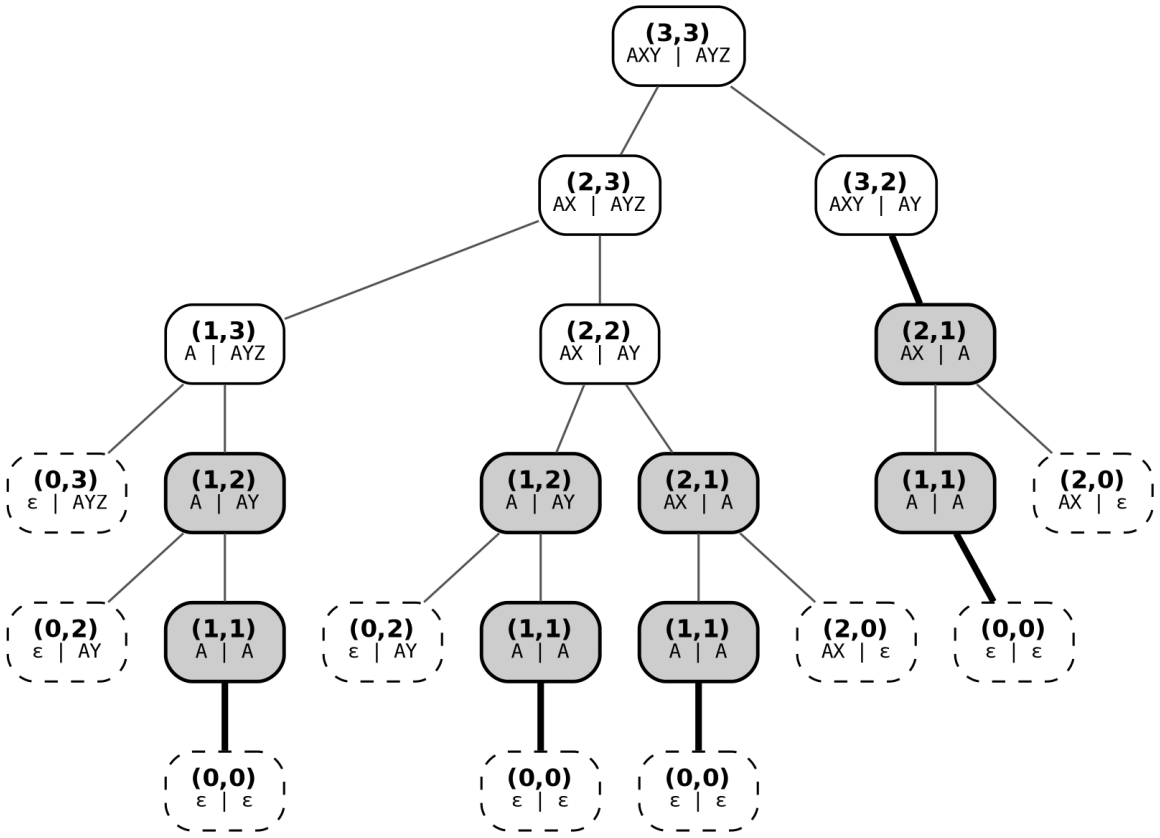
A recorrência vira praticamente direto uma função recursiva. A primeira chamada é LCS-Recursivo(X, Y, m, n):

```
LCS-Recursivo(X, Y, i, j)
1  se i = 0 ou j = 0
2      retorne 0                                ▷ prefixo vazio
3  se X[i] = Y[j]
4      retorne LCS-Recursivo(X, Y, i-1, j-1) + 1
5  senão
6      retorne max( LCS-Recursivo(X, Y, i-1, j),
                    LCS-Recursivo(X, Y, i, j-1) )
```

3.3 Exemplo

Para ver o algoritmo funcionando, montamos a árvore de recursão completa, conforme vimos em aula, para X = `AXY` e Y = `AYZ` (Figura 1). Cada nó (i, j) é uma chamada da função, e embaixo aparecem os prefixos que ela compara (ε é o prefixo vazio). Na raiz, o último caractere de X é Y e o de Y é Z, que são diferentes, então a chamada se divide em (2, 3) e (3, 2). Em (3, 2) os dois últimos caracteres são Y, então há um casamento e a recursão segue direto para (2, 1).

Figura 1 - Árvore de recursão da força bruta para X = AXY e Y = AYZ (22 chamadas)



O resultado é c(3, 3) = 2 (a LCS é AY), e para chegar nele o algoritmo fez 22 chamadas. O problema fica claro nos nós cinza: o subproblema (1, 1) foi resolvido quatro vezes, e (1, 2) e (2, 1) duas vezes cada. Com strings de 3 caracteres isso não pesa muito, mas esse retrabalho cresce muito rápido conforme as strings aumentam.

3.4 Complexidade

O custo da força bruta depende bastante de quais são as strings, e não só do tamanho delas. O melhor caso é quando as duas são iguais: todo caractere casa, a recursão vai só pela diagonal e são feitas apenas n + 1 chamadas. O pior caso é quando não há nenhum caractere em comum, porque aí toda chamada se divide em duas. Nessa situação o número de chamadas é $2 \cdot C(m + n, n) - 1$, em que C é o coeficiente binomial. De forma geral, como cada chamada gera no máximo duas outras e a recursão tem profundidade de no máximo m + n, o tempo é $O(2^{m+n})$, ou seja, exponencial.

Cada caractere a mais em cada string multiplica o trabalho por quase 4. E o mais estranho é que existem só (m + 1)(n + 1) subproblemas diferentes. Com n = 15, por exemplo, são 256 pares (i, j), mas a força bruta faz mais de 310 milhões de chamadas porque fica resolvendo os mesmos pares de novo e de novo. Em relação à memória, o gasto é O(m + n), que é a profundidade máxima da pilha de recursão.

4 ALGORITMO COM PROGRAMAÇÃO DINÂMICA

4.1 Ideia

A Programação Dinâmica resolve exatamente o problema que vimos na força bruta. Ela pode ser usada aqui porque a LCS tem duas propriedades. A primeira é a subestrutura ótima: a LCS das strings inteiras é montada a partir da LCS de prefixos delas, como a recorrência da Seção 3.1 mostrou. A segunda é a sobreposição de subproblemas, que ficou bem visível na Figura 1, onde os mesmos pares (i, j) apareciam várias vezes.

A ideia, então, é resolver cada subproblema uma vez só e guardar a resposta em uma tabela C[0..m][0..n]. Preenchemos essa tabela de baixo para cima (*bottom-up*), linha por linha, da esquerda para a direita. Dessa forma, quando chegamos na célula C[i][j], as três células de que ela depende (a da diagonal C[i-1][j-1], a de cima C[i-1][j] e a da esquerda C[i][j-1]) já estão calculadas.

4.2 Algoritmo

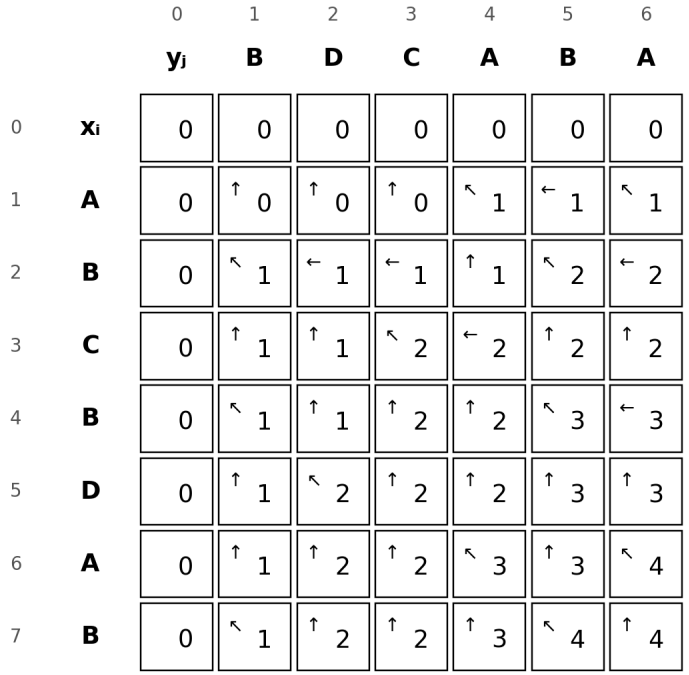
```
LCS-PD(X, Y)
 1  m ← |X|;  n ← |Y|
 2  seja C[0..m, 0..n] uma nova tabela
 3  para i ← 0 até m:  C[i, 0] ← 0          ▷ coluna 0: Y vazio
 4  para j ← 0 até n:  C[0, j] ← 0          ▷ linha 0: X vazio
 5  para i ← 1 até m
 6      para j ← 1 até n                    ▷ uma iteração por célula
 7          se X[i] = Y[j]
 8              C[i, j] ← C[i-1, j-1] + 1
 9          senão
10              C[i, j] ← max(C[i-1, j], C[i, j-1])
11  retorne C[m, n]
```

As linhas 3 e 4 preenchem os casos base, e o laço duplo das linhas 5 a 10 roda exatamente m·n vezes. Em cada volta ele faz uma comparação e, no máximo, um cálculo de máximo, então cada iteração custa tempo constante.

4.3 Exemplo passo a passo

A Figura 2 mostra a tabela preenchida para o exemplo clássico, com X = `ABCBDAB` nas linhas e Y = `BDCABA` nas colunas. Em cada célula aparece o valor de C[i][j] e uma setinha indicando de onde ele veio: da diagonal (quando os caracteres casam), de cima ou da esquerda.

Figura 2 - Tabela da Programação Dinâmica para X = ABCBDAB e Y = BDCABA



		0	1	2	3	4	5	6
		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	↑ 0	↑ 0	↑ 0	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	↑ 1	↖ 2	← 2
3	C	0	↑ 1	↑ 1	↖ 2	← 2	↑ 2	↑ 2
4	B	0	↖ 1	↑ 1	↑ 2	↑ 2	↖ 3	← 3
5	D	0	↑ 1	↖ 2	↑ 2	↑ 2	↑ 3	↑ 3
6	A	0	↑ 1	↑ 2	↑ 2	↖ 3	↑ 3	↖ 4
7	B	0	↖ 1	↑ 2	↑ 2	↑ 3	↖ 4	↑ 4

Assim, chegamos em C[7][6] = 4 com 7 × 6 = 42 iterações. Para o mesmo exemplo, a força bruta precisou de 152 chamadas.

4.4 Complexidade

Como o algoritmo faz exatamente m·n iterações de custo constante, o tempo é Θ(m·n), e a tabela ocupa Θ(m·n) de memória. Uma diferença importante em relação à força bruta é que o número de iterações não depende do conteúdo das strings, só do tamanho delas.

5 DESCRIÇÃO DA IMPLEMENTAÇÃO

5.1 Visão geral

Fizemos a implementação em Java e não usamos nenhuma biblioteca externa. O programa calcula o tamanho da LCS com a Programação Dinâmica, que é a nossa versão principal, e também com a força bruta, para podermos comparar. Para cada algoritmo ele conta as iterações e mede o tempo de execução.

Dá para usar o programa de quatro jeitos: passando as duas strings na linha de comando, lendo as strings de um arquivo, pelo menu interativo ou rodando a bateria de testes. O código ficou dividido nas classes da Tabela 2.

Tabela 2 - Classes da implementação

Classe	O que faz
`LCSProgramacaoDinamica`	Programação Dinâmica bottom-up com tabela (versão principal). Conta as iterações, reconstrói uma LCS e imprime a tabela.
`LCSForcaBruta`	Algoritmo recursivo da Seção 3.2. Conta as chamadas recursivas.

Classe	O que faz
`Medidor`	Mede o tempo médio de execução de um algoritmo.
`BateriaTestes`	Guarda os 11 casos de teste, imprime a tabela-resumo e grava um arquivo CSV com os resultados.
`Main`	Ponto de entrada: trata os argumentos, a leitura de arquivo e o menu.

5.2 Contagem de iterações

Como os dois algoritmos funcionam de jeitos diferentes, a "iteração" de cada um também é diferente. Na força bruta, contamos cada chamada da função recursiva, inclusive as que caem no caso base. Na Programação Dinâmica, contamos cada volta do laço interno, ou seja, cada célula C[i][j] calculada, o que sempre dá m·n.

Como a força bruta fica inviável muito rápido, o programa só executa ela quando m + n ≤ 32. Se usarmos a opção `--completo`, esse limite sobe para m + n ≤ 60, e aí ela roda também nos casos 08 e 09, que levam alguns minutos. Nos casos maiores que isso ela não é executada.

5.3 Medição do tempo

Para medir o tempo usamos `System.nanoTime()`. O problema é que vários casos levam menos de um microssegundo, e a JVM ainda vai compilando e otimizando o código enquanto ele roda (compilação JIT), então uma medição só não seria confiável. Por isso, a classe `Medidor` primeiro roda o algoritmo repetidas vezes por uns 100 ms para "aquecer" a JVM, e depois roda de novo por pelo menos 200 ms, calculando o tempo médio de cada execução. Quando uma única execução já passa de 0,5 s (caso da força bruta nos testes maiores), usamos o tempo dela direto.

5.4 Casos de teste

A bateria de testes (`--testes`) tem 11 casos. Os seis primeiros (01 a 06, Tabela 1) são simples e dá para conferir à mão. Os outros cinco são mais elaborados: o 07 é o pior caso da força bruta (15 × 15, sem nada em comum), o 08 usa frases com espaços (20 × 21), o 09 usa duas sequências de DNA (29 × 28), o 10 é um DNA aleatório de 1.000 × 1.000 e o 11 usa letras aleatórias com 5.000 × 5.000. As strings aleatórias são geradas sempre com a mesma semente, então todas as execuções usam exatamente as mesmas entradas.

5.5 Como compilar e executar

Com o JDK instalado, basta entrar na pasta `codigo` e usar os scripts:

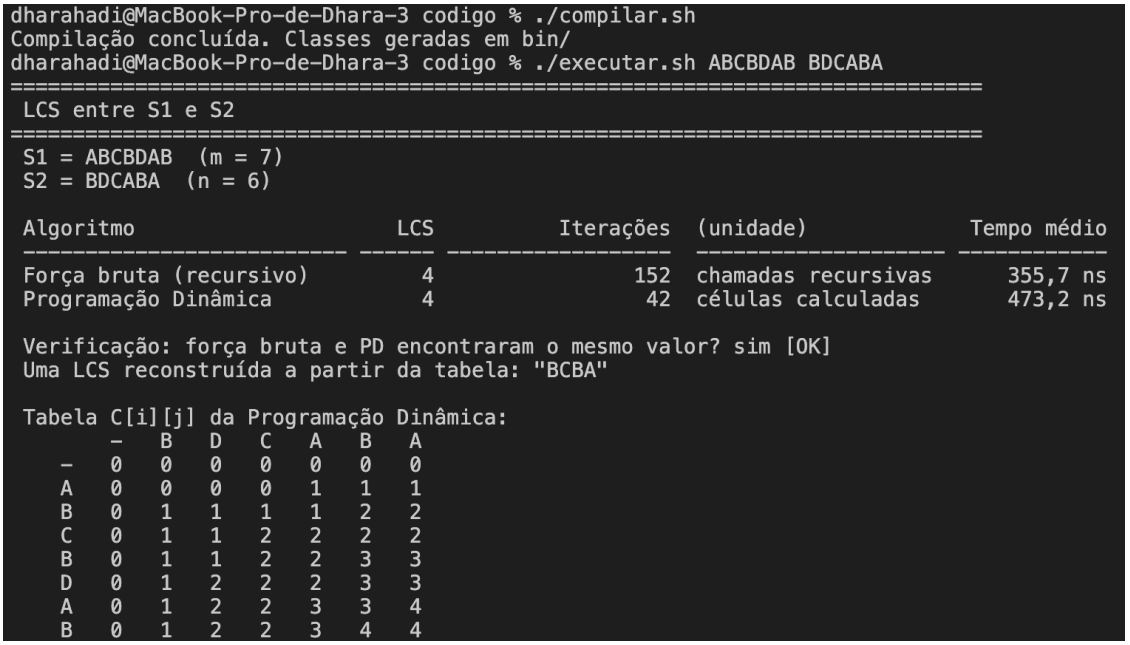
```
./compilar.sh                                  # compila para a pasta bin/
./executar.sh                                  # menu interativo
./executar.sh ABCBDAB BDCABA                   # LCS entre duas strings
./executar.sh --arquivo testes/caso_dna_livro.txt
./executar.sh --testes                         # bateria de testes
./executar.sh --testes --completo              # força bruta até m+n = 60
```

6 EXEMPLOS DE EXECUÇÃO

Nesta seção estão algumas capturas de tela do programa rodando no terminal. Começamos pelos casos mais simples e terminamos com a bateria de testes completa.

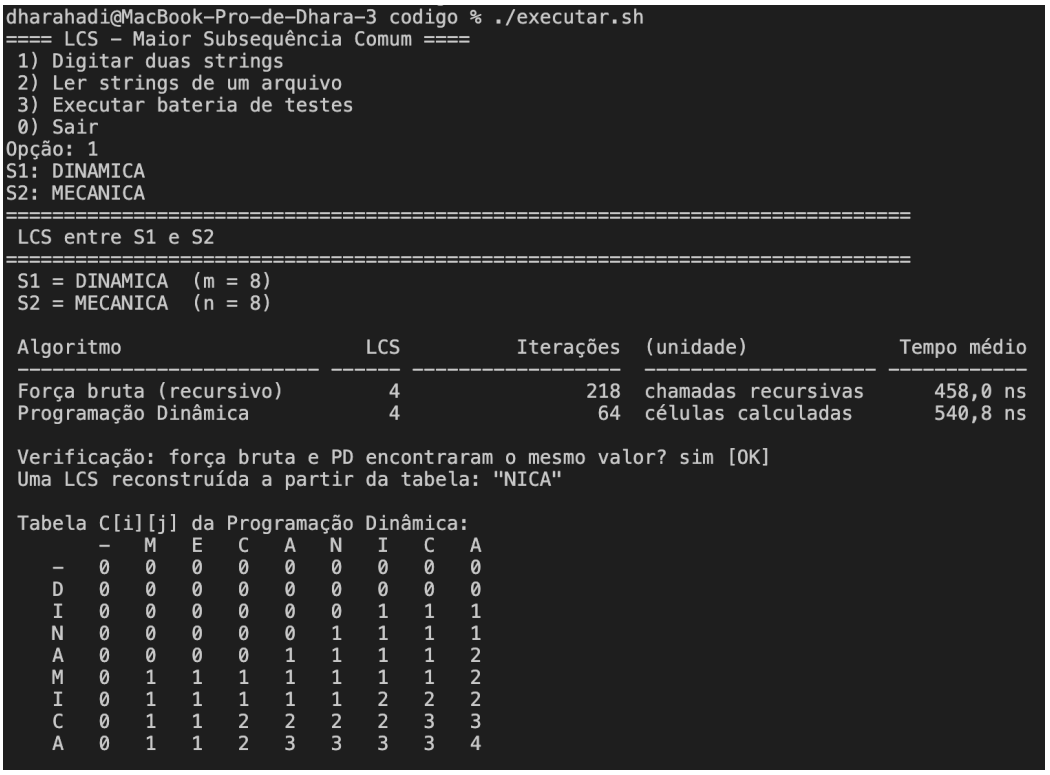
Na Figura 3 aparece a compilação e a execução com o exemplo clássico. Os dois algoritmos chegam em L = 4, mas a força bruta precisa de 152 chamadas, enquanto a PD faz 42 iterações (7 × 6). Como as strings são pequenas, o programa também mostra a tabela C, que é a mesma da Figura 2.

Figura 3 - Compilação e execução com X = ABCBDAB e Y = BDCABA



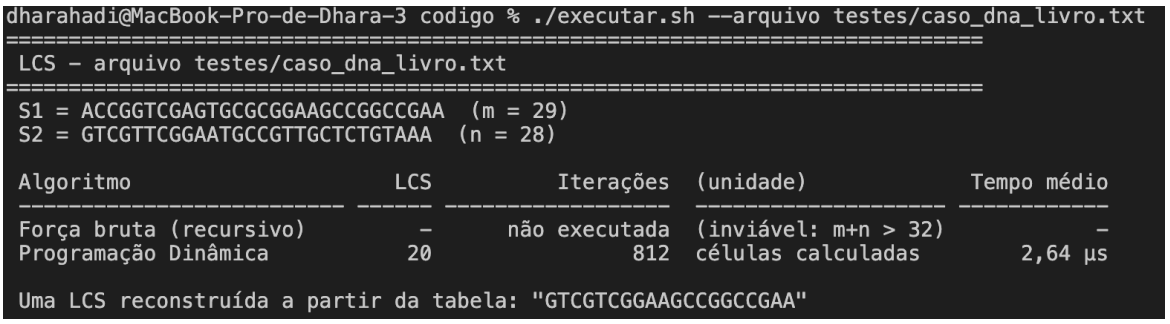
Na Figura 4 usamos o menu interativo para digitar duas palavras, DINAMICA e MECANICA. O resultado foi L = 4, com a LCS `NICA`. A força bruta fez 218 chamadas e a PD, 64 iterações.

Figura 4 - Menu interativo com as palavras DINAMICA e MECANICA



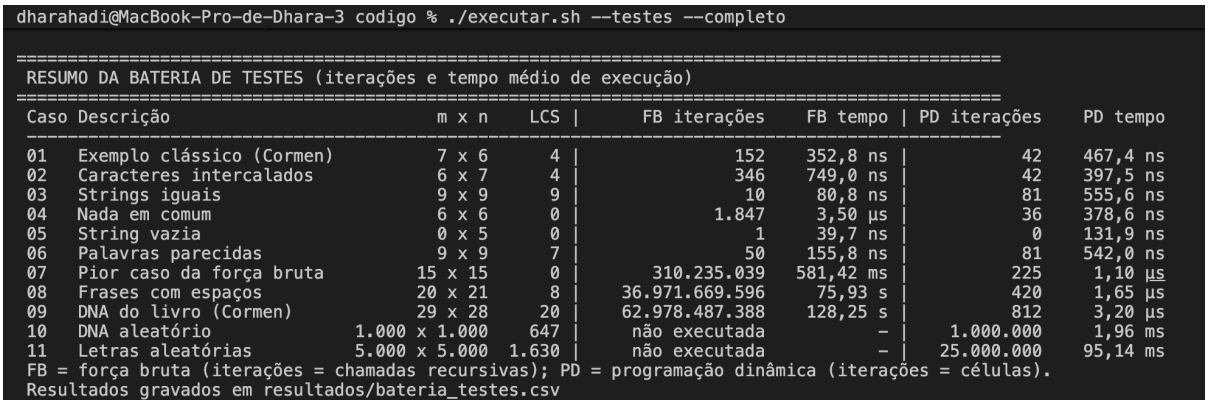
A Figura 5 mostra a leitura de duas sequências de DNA a partir de um arquivo. Aqui m + n = 57, então no modo normal o programa nem tenta rodar a força bruta. A PD resolve o caso com 812 iterações, em poucos microssegundos, e encontra L = 20.

Figura 5 - Leitura de arquivo com duas sequências de DNA



Por último, a Figura 6 mostra o resumo da bateria de testes rodada com `--completo`, com as iterações e os tempos dos 11 casos.

Figura 6 - Resumo da bateria de testes



7 CONTABILIZAÇÃO DE ITERAÇÕES E TEMPO DE EXECUÇÃO

As Tabelas 3 e 4 trazem o número de iterações e o tempo médio de cada algoritmo em cada teste, obtidos com `./executar.sh --testes --completo`. Nos casos 10 e 11 a força bruta não foi executada, porque levaria tempo demais.

Os testes foram rodados em um MacBook Pro com chip Apple M1, 8 GB de memória RAM, macOS e Java OpenJDK 21.0.7. Os tempos mudam um pouco de uma execução para outra e de uma máquina para outra, então o que mais importa aqui é a diferença entre os algoritmos, e não os valores exatos.

Tabela 3 - Número de iterações de cada teste

Caso	m × n	L	Força bruta (chamadas)	PD (células)
01	7 × 6	4	152	42
02	6 × 7	4	346	42
03	9 × 9	9	10	81
04	6 × 6	0	1.847	36
05	0 × 5	0	1	0
06	9 × 9	7	50	81
07	15 × 15	0	310.235.039	225
08	20 × 21	8	36.971.669.596	420
09	29 × 28	20	62.978.487.388	812
10	1.000 × 1.000	647	não executada	1.000.000
11	5.000 × 5.000	1.630	não executada	25.000.000

Tabela 4 - Tempo médio de execução de cada teste

Caso	m × n	Força bruta	Programação Dinâmica
01	7 × 6	352,8 ns	467,4 ns
02	6 × 7	749,0 ns	397,5 ns
03	9 × 9	80,8 ns	555,6 ns
04	6 × 6	3,50 µs	378,6 ns
05	0 × 5	39,7 ns	131,9 ns

Caso	m × n	Força bruta	Programação Dinâmica
06	9 × 9	155,8 ns	542,0 ns
07	15 × 15	581,42 ms	1,10 µs
08	20 × 21	75,93 s	1,65 µs
09	29 × 28	128,25 s	3,20 µs
10	1.000 × 1.000	não executada	1,96 ms
11	5.000 × 5.000	não executada	95,14 ms